FinHedge AI

High-Level Design & System Architecture

DA5402 - MLOps in Practice
Indian Institute of Technology Madras
April 2026

Author: Siddharth Chauhan
Roll No.: CH21B103
Course: DA5402 MLOps
Project: FinHedge AI - Stock Prediction & Hedging Platform

This document describes the complete system architecture of FinHedge AI, covering component design, data flow, infrastructure topology, and design decisions.

0 Contents

1 Overview **2**

2 High-Level Architecture **2**
2.1 Component Map . . . 2
2.2 Container Summary . . . 3

3 Data Layer Architecture **3**
3.1 Data Flow . . . 3
3.2 Feature Engineering Pipeline . . . 4
3.3 DVC Pipeline Stages . . . 4

4 ML Model Architecture **4**
4.1 LSTM Model . . . 4
4.2 GRU Model . . . 5
4.3 Training Configuration . . . 5
4.4 Evaluation Metrics . . . 6

5 Backend API Architecture **6**
5.1 API Layer Design . . . 7
5.2 Key Endpoints . . . 7

6 Observability Stack **8**
6.1 Prometheus Metrics Schema . . . 8
6.2 Grafana Dashboard Layout . . . 8

7 CI/CD Pipeline **9**
7.1 GitHub Actions Workflow . . . 10
7.2 Job Breakdown . . . 10

8 Network & Infrastructure Topology **10**
8.1 Docker Network . . . 10
8.2 Volume Mounts . . . 11

9 Airflow DAG Design **11**

10 Design Decisions & Trade-offs **12**

Appendix - Folder Structure **12**

1 Overview

FinHedge AI is a production-grade MLOps platform for stock price forecasting and portfolio hedging. Built for the DA5402 course at IIT Madras, it demonstrates end-to-end MLOps practices - from automated data ingestion through model training, serving, drift monitoring, and CI/CD.

Design Goals:

- Modular microservices - each concern has a single service boundary
- Reproducibility - DVC tracks every data/model artefact; MLflow logs every run
- Observability - Prometheus + Grafana give real-time metrics; drift detection triggers retraining
- One-command deployment - `docker compose up` brings the full stack online
- CI/CD - GitHub Actions lint, test, and smoke-test on every push

2 High-Level Architecture

2.1 Component Map

The system comprises six layers, each containerised and communicating over a shared Docker network (`finhedge_net`):

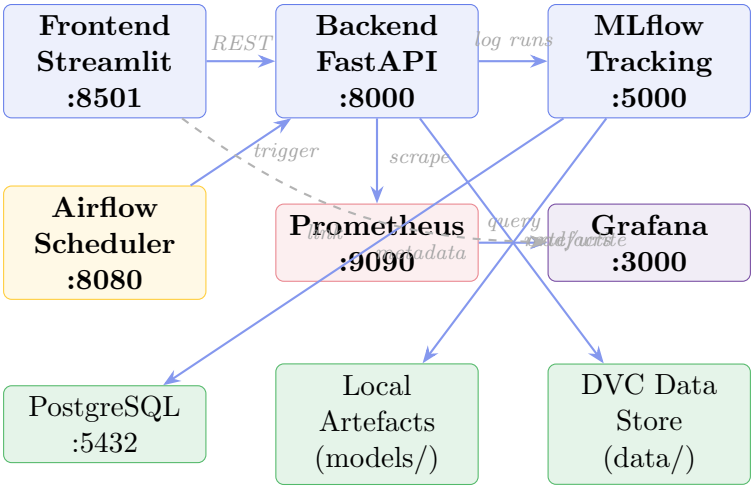
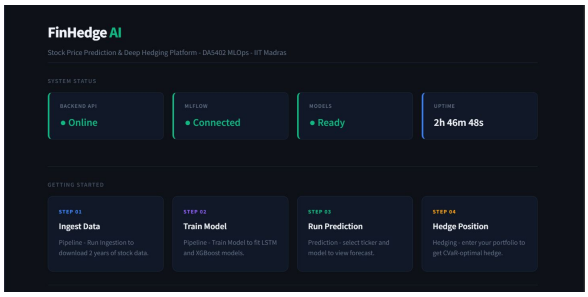


Figure 1: FinHedge AI system component map

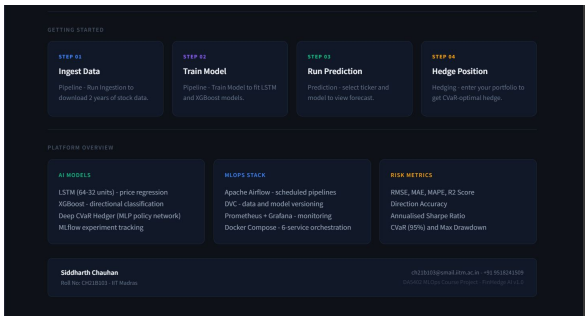
2.2 Container Summary

Table 1: Docker services in `docker-compose.yml`

Service	Image	Port(s)	Responsibility
`backend`	Python 3.10 / FastAPI	8000	REST API, model inference, Prometheus metrics
`frontend`	Python 3.10 / Streamlit	8501	Web UI (5 pages)
`mlflow`	Python + MLflow	5000	Experiment tracking, model registry
`postgres`	PostgreSQL 15	5432	MLflow metadata backend
`prometheus`	prom/prometheus	9090	Metric collection & alerting rules
`grafana`	grafana/grafana	3000	Dashboards (10 panels)
`airflow`	apache/airflow:2.9	8080	DAG scheduling (SequentialExecutor)
`redis`	redis:7-alpine	6379	(Reserved for task queue)



(a) System dashboard - homepage



(b) Stock price chart with predictions

Figure 2: FinHedge AI - Streamlit frontend overview

3 Data Layer Architecture

3.1 Data Flow

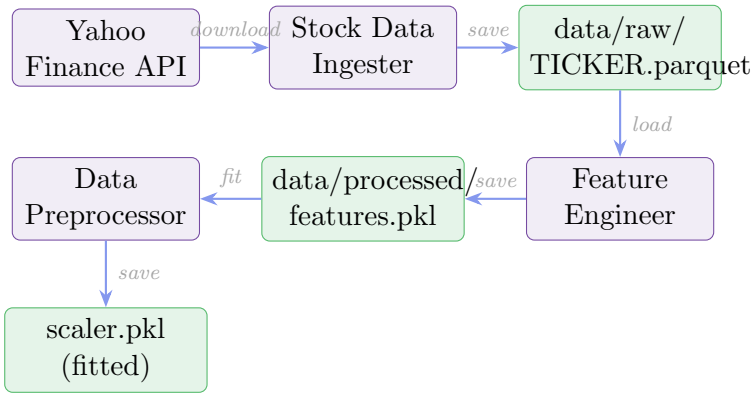


Figure 3: Data ingestion and preprocessing flow

3.2 Feature Engineering Pipeline

Nineteen technical indicators are computed from OHLCV data:

Table 2: Feature engineering: indicator groups

Group	Features	Purpose
Trend	SMA-20, EMA-12, EMA-26, MACD, Signal	Trend direction
Momentum	RSI-14, ROC-10, Stochastic-K/D	Overbought / oversold
Volatility	Bollinger Bands width, ATR-14, Hist. Vol-20	Risk sizing
Volume	Vol/MA ratio, OBV	Liquidity
Returns	Return-1d, Ret-mean-5, Ret-std-20	Statistical features

3.3 DVC Pipeline Stages

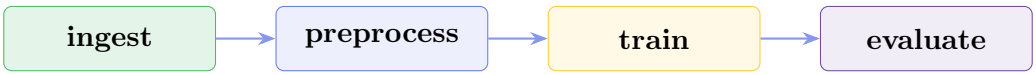
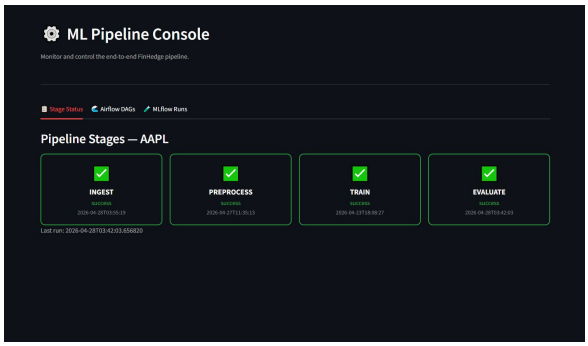
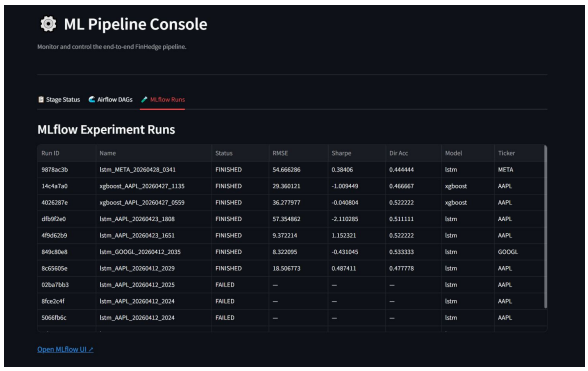


Figure 4: DVC pipeline: four sequential stages tracked in `dvc.yaml`

Each stage is tracked by DVC; outputs are hashed and cached so unchanged stages are skipped on subsequent runs.



(a) Pipeline stage status in UI



(b) DVC pipeline progress

Figure 5: Data pipeline execution views

4 ML Model Architecture

4.1 LSTM Model

The primary model is a stacked LSTM network for time-series forecasting:

Listing 1: LSTM architecture (backend/core/models/lstm_model.py)

```
class LSTMModel(nn.Module):
    def __init__(self, input_size, hidden_size=128,
                 num_layers=2, dropout=0.2):
        super().__init__()
```

```
        self.lstm = nn.LSTM(
            input_size, hidden_size,
            num_layers=num_layers,
            batch_first=True,
            dropout=dropout
        )
        self.fc = nn.Linear(hidden_size, 1)

    def forward(self, x):
        out, _ = self.lstm(x)
        return self.fc(out[:, -1, :])    # last time step
```

4.2 GRU Model

A GRU variant is available as a lightweight alternative:

Listing 2: GRU model (backend/core/models/gru_model.py)

```
class GRUModel(nn.Module):
    def __init__(self, input_size, hidden_size=128,
                 num_layers=2, dropout=0.2):
        super().__init__()
        self.gru = nn.GRU(
            input_size, hidden_size,
            num_layers=num_layers,
            batch_first=True,
            dropout=dropout
        )
        self.fc = nn.Linear(hidden_size, 1)
```

4.3 Training Configuration

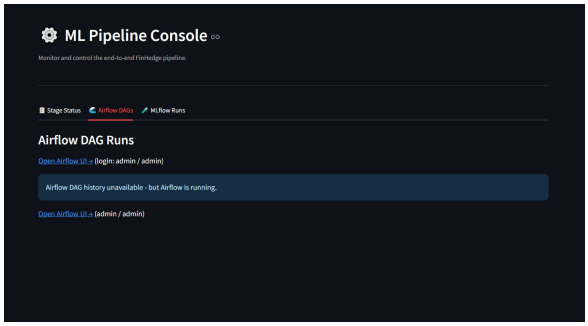
Table 3: Default hyperparameters

Parameter	Default	Range
Sequence length	60 days	20–120
Hidden units	128	64–256
LSTM layers	2	1–4
Dropout	0.2	0.1–0.5
Learning rate	0.001	1e-4 – 1e-2
Batch size	32	16–128
Epochs	50	10–200
Optimiser	Adam	-
Loss	MSELoss	-

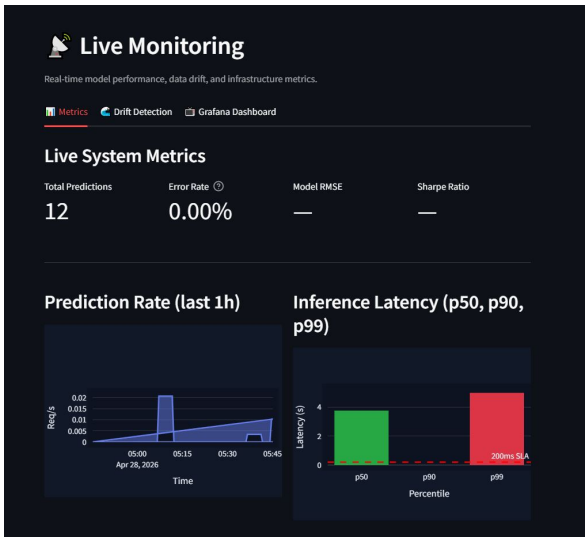
4.4 Evaluation Metrics

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2}, \qquad \text{MAPE} = \frac{100}{N}\sum_{i=1}^{N}\left|\frac{y_i - \hat{y}_i}{y_i}\right|$$

$$\text{Sharpe} = \frac{E[r_t - r_f]}{\sigma(r_t - r_f)}, \qquad \text{Direction Acc.} = \frac{\text{correct sign predictions}}{N}$$

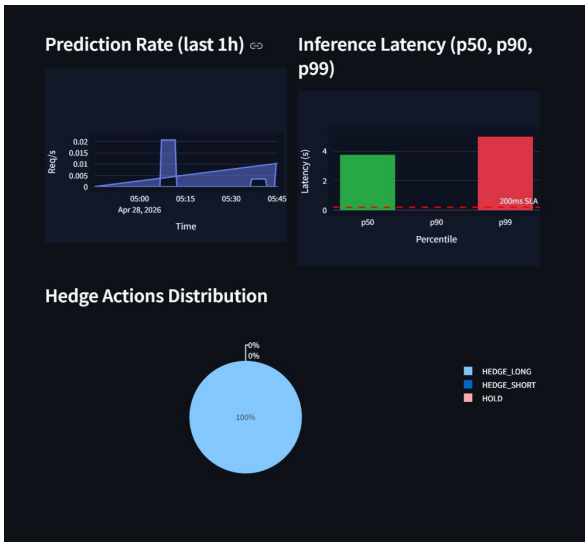


(a) MLflow experiment tracking

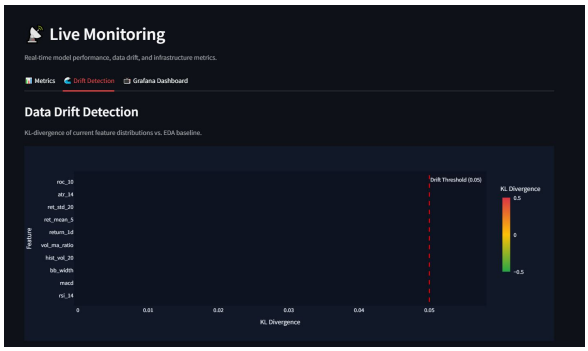


(b) Training metrics dashboard

Figure 6: Model training and experiment management



(a) MLflow run details



(b) Model artefacts in registry

Figure 7: MLflow model tracking views

5 Backend API Architecture

5.1 API Layer Design

The FastAPI backend exposes a RESTful interface organised into four route groups:

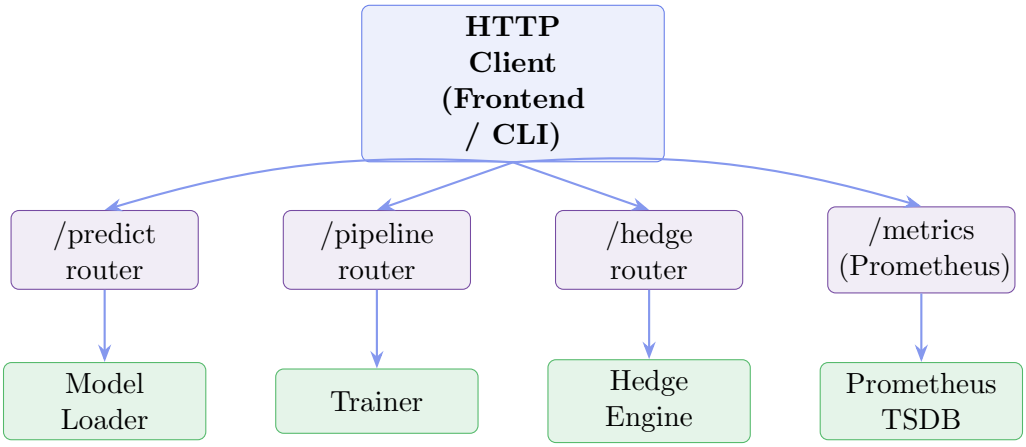
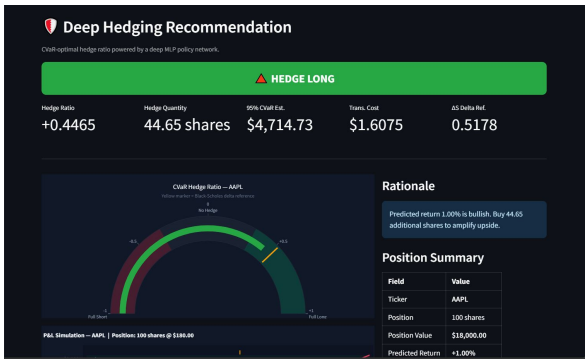


Figure 8: Backend API routing architecture

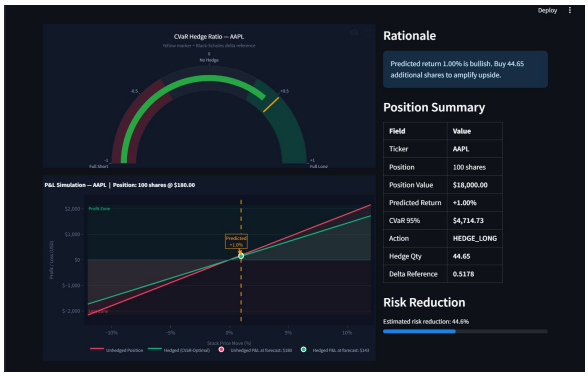
5.2 Key Endpoints

Table 4: REST API endpoint reference

Method	Path	Auth	Description
POST	`/predict/price`	-	Single-step price prediction
POST	`/predict/batch`	-	Batch predictions
GET	`/predict/history`	-	Prediction history log
GET	`/pipeline/status/t`	-	DVC stage statuses
POST	`/pipeline/trigger`	-	Async stage execution
POST	`/pipeline/train`	-	Full synchronous training run
GET	`/pipeline/runs`	-	MLflow experiment runs
GET	`/pipeline/data/t/ohlcv`	-	Raw OHLCV data
POST	`/pipeline/drift/compute`	-	Compute drift scores
POST	`/hedge/recommend`	-	Hedging strategy recommendation
GET	`/health`	-	Liveness probe
GET	`/metrics`	-	Prometheus scrape endpoint



(a) FastAPI interactive documentation



(b) API endpoint explorer

Figure 9: FastAPI Swagger UI (`/docs`)

6 Observability Stack

6.1 Prometheus Metrics Schema

Table 5: Prometheus gauge and counter definitions

Metric Name	Type	Labels
`finhedge_prediction_requests_total`	Counter	ticker, model
`finhedge_prediction_errors_total`	Counter	ticker, model
`finhedge_prediction_latency_seconds`	Histogram	ticker, model
`finhedge_model_rmse`	Gauge	ticker, model
`finhedge_model_sharpe`	Gauge	ticker, model
`finhedge_model_direction_acc`	Gauge	ticker, model
`finhedge_data_drift_score`	Gauge	ticker, feature
`finhedge_drift_alert`	Gauge	ticker
`finhedge_hedge_requests_total`	Counter	ticker, action
`finhedge_pipeline_runs_total`	Counter	stage, status
`finhedge_training_duration_seconds`	Histogram	ticker, model

6.2 Grafana Dashboard Layout

The pre-provisioned `finhedge.json` dashboard contains 10 panels arranged in four rows as shown below:

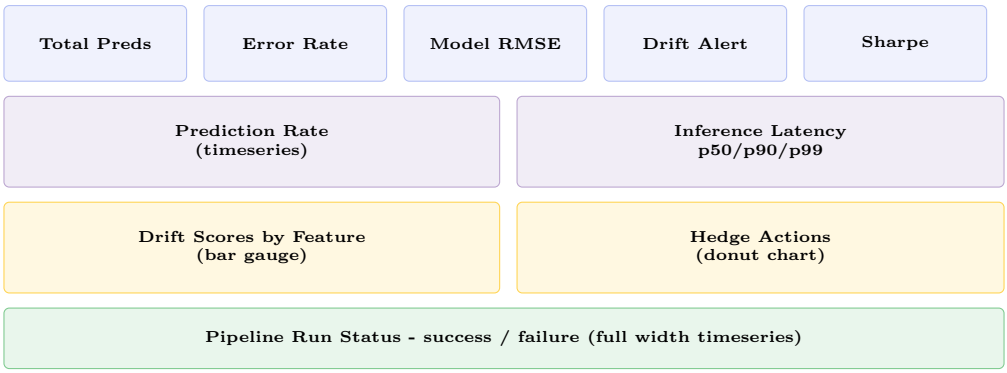
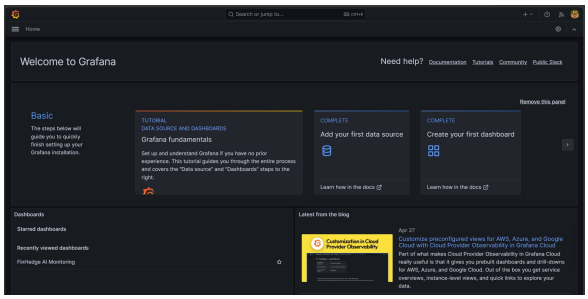
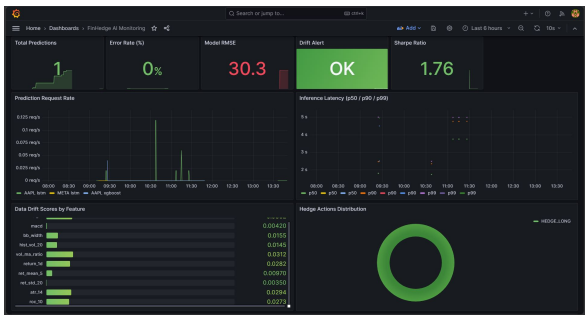


Figure 10: Grafana dashboard layout - 10 panels, 4 rows

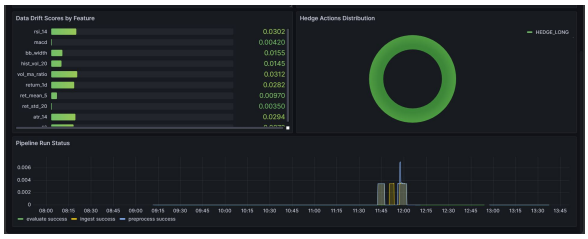


(a) Grafana dashboard - stat panels

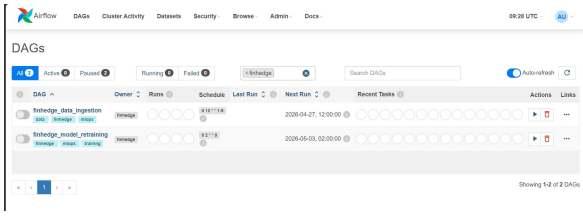


(b) Grafana - timeseries and bargauge

Figure 11: Grafana monitoring dashboards



(a) Prometheus metrics endpoint



(b) Streamlit monitoring page

Figure 12: Live observability views

7 CI/CD Pipeline

7.1 GitHub Actions Workflow

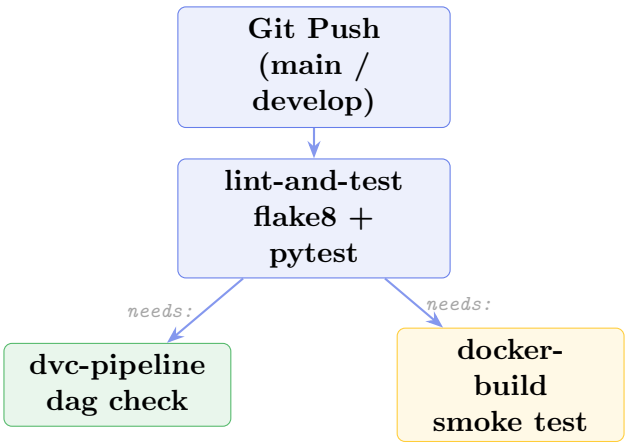


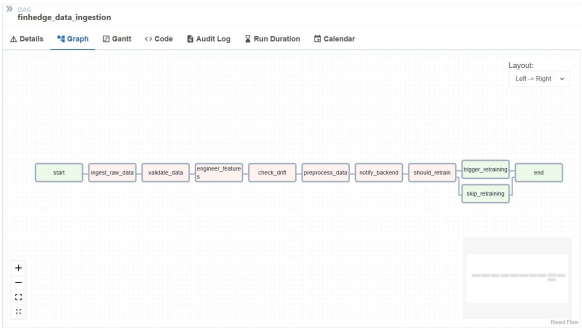
Figure 13: GitHub Actions CI/CD: three-job workflow

7.2 Job Breakdown

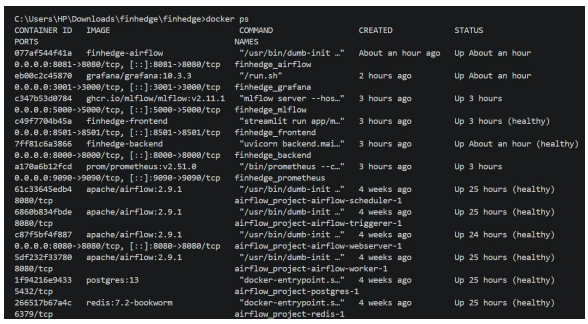
`lint-and-test` Installs Python 3.10, runs `flake8` (max-line-length=100), `black` format-check, `isort`, and `pytest backend/tests/`. Uploads `test_report.txt` as a build artefact.

`dvc-pipeline` Installs DVC and runs `dvc dag --md` to verify the pipeline DAG is consistent with `dvc.yaml`.

`docker-build` Builds `finhedge-backend:ci` and `finhedge-frontend:ci` images, then smoke-tests the backend container by polling `/health`.



(a) GitHub Actions - workflow runs



```
C:\Users\HP\Downloads\finhedge\finhedge>docker ps
CONTAINER ID   IMAGE                          COMMAND                  CREATED             STATUS
PORTS                                         NAMES
077af544f41a   finhedge-airflow               "/usr/bin/dumb-init …"   About an hour ago   Up About an hour
0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp   finhedge_airflow
eb00c2c45870   grafana/grafana:10.3.3         "/run.sh"                2 hours ago         Up About an hour
0.0.0.0:3001->3000/tcp, [::]:3001->3000/tcp   finhedge_grafana
c347b53d0784   ghcr.io/mlflow/mlflow:v2.11.1  "mlflow server --hos…"   3 hours ago         Up 3 hours
0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   finhedge_mlflow
c49f7704b45a   finhedge-frontend              "streamlit run app/m…"   3 hours ago         Up 3 hours (healthy)
0.0.0.0:8501->8501/tcp, [::]:8501->8501/tcp   finhedge_frontend
7ff81c6a3866   finhedge-backend               "uvicorn backend.mai…"   3 hours ago         Up About an hour (healthy)
0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp   finhedge_backend
a170a6b12fcd   prom/prometheus:v2.51.0        "/bin/prometheus --c…"   3 hours ago         Up 3 hours
0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp   finhedge_prometheus
61c33645edb4   apache/airflow:2.9.1           "/usr/bin/dumb-init …"   4 weeks ago         Up 25 hours (healthy)
8080/tcp                                      airflow_project-airflow-scheduler-1
6860a834fbde   apache/airflow:2.9.1           "/usr/bin/dumb-init …"   4 weeks ago         Up 25 hours (healthy)
8080/tcp                                      airflow_project-airflow-triggerer-1
c87f5bf4f887   apache/airflow:2.9.1           "/usr/bin/dumb-init …"   4 weeks ago         Up 24 hours (healthy)
0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp   airflow_project-airflow-webserver-1
5df232f33780   apache/airflow:2.9.1           "/usr/bin/dumb-init …"   4 weeks ago         Up 25 hours (healthy)
8080/tcp                                      airflow_project-airflow-worker-1
1f04216e9433   postgres:13                    "docker-entrypoint.s…"   4 weeks ago         Up 25 hours (healthy)
5432/tcp                                      airflow_project-postgres-1
266517b67a4c   redis:7.2-bookworm             "docker-entrypoint.s…"   4 weeks ago         Up 25 hours (healthy)
6379/tcp                                      airflow_project-redis-1
```

(b) CI job details

Figure 14: GitHub Actions CI/CD pipeline

8 Network & Infrastructure Topology

8.1 Docker Network

All services communicate on the bridge network `finhedge_net`. DNS resolution uses container service names (e.g., `prometheus:9090`, `mlflow:5000`).

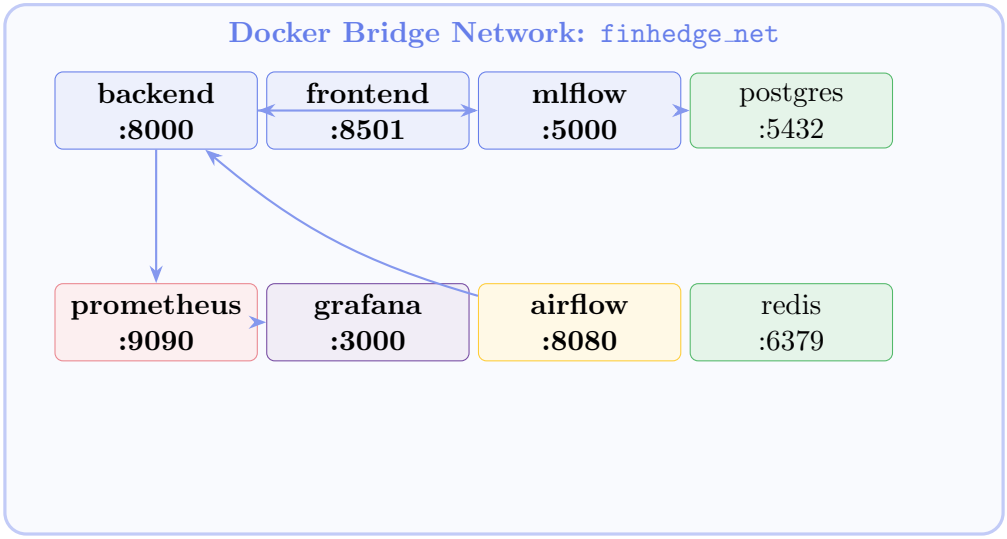


Figure 15: Docker network topology - all services on `finhedge_net`

8.2 Volume Mounts

Table 6: Persistent volumes in `docker-compose.yml`

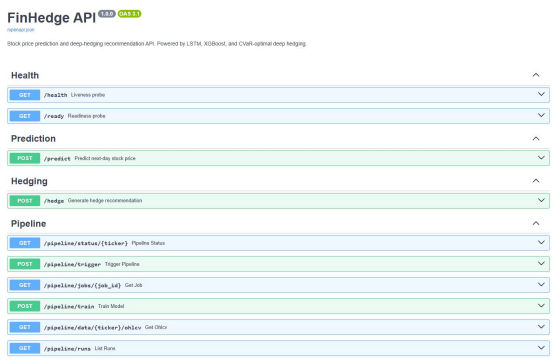
Host Path	Container Path	Service
`./data`	`/app/data`	backend, airflow
`./models`	`/app/models`	backend
`./metrics`	`/app/metrics`	backend
`./mlruns`	`/mlruns`	mlflow
`./grafana/dashboards`	`/var/lib/grafana/dashboards`	grafana
`./grafana/provisioning`	`/etc/grafana/provisioning`	grafana
`./prometheus/prometheus.yml`	`/etc/prometheus/prometheus.yml`	prometheus

9 Airflow DAG Design

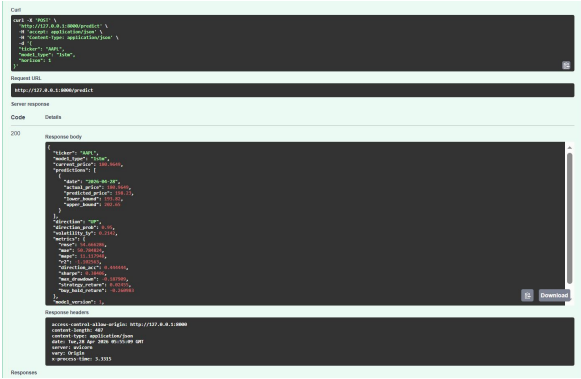
The Airflow DAG `finhedge_pipeline` orchestrates the end-to-end pipeline on a configurable schedule (default: daily at midnight UTC).

Figure 16: Airflow DAG: `finhedge_pipeline` task chain

Each task calls the corresponding REST endpoint on the backend service, so the pipeline logic lives in the backend and Airflow acts purely as a scheduler/orchestrator.



(a) Airflow DAG graph view



(b) Airflow task logs

Figure 17: Apache Airflow orchestration

10 Design Decisions & Trade-offs

Table 7: Key architectural decisions

Decision	Chosen Approach	Rationale
Airflow executor	SequentialExecutor + SQLite	Avoids PostgreSQL dependency for Airflow; sufficient for course demo scale
Model store	MLflow Model Registry + local filesystem	Simple setup; registry provides versioning without external object store
In-process job tracker	Python dict `_jobs`	Zero-dependency; production would use Redis + Celery
Drift detection	KL-divergence vs. EDA baseline	Interpretable; computed on demand via `/drift/compute`
Prometheus scrape	Pull model (backend exposes `/metrics`)	Standard pattern; works with Docker DNS without extra config
Grafana embedding	Browser link (not iframe)	Chrome blocks cross-origin `localhost` iframes in production

10 Appendix - Folder Structure

Listing 3: Project repository layout

```
finhedge/
          backend/
                 api/
```

```
                        routes/          # predict, pipeline, hedge, health
                        schemas.py       # Pydantic models
                core/
                        data/            # ingestion, features, preprocessing
                        models/          # lstm_model.py, gru_model.py
                        training/        # trainer.py
                        monitoring/      # metrics.py (Prometheus counters/gauges)
                tests/                # pytest unit tests
                Dockerfile
        frontend/
                app/
                        main.py
                        pages/           # 01_Dashboard, 02_Hedging, 03_Pipeline,
                                      # 04_Monitoring, 05_MLflow
                Dockerfile
        grafana/
                dashboards/finhedge.json
                provisioning/
        prometheus/prometheus.yml
        dags/finhedge_pipeline.py
        dvc.yaml
        .github/workflows/ci.yml
        docker-compose.yml
```